Guía 2 de 4 — Columnas nuevas, fechas y limpieza

DPL1046 · Clase 5 · Para repasar en casa, a tu ritmo.

Los datos reales casi nunca vienen listos para analizar. En esta guía repasas cómo **crear columnas**, **trabajar con fechas** y **limpiar** valores vacíos y filas repetidas, hasta responder una pregunta de negocio con un número.

> Antes de empezar, haz tu rutina de inicio (Guía 1 de la Clase 2) con el `.venv` activado.

---

1. Crear una columna nueva

En Excel escribes `=B2+C2` y arrastras hacia abajo. En pandas es **una sola línea para toda la columna**:

```
items = pd.read_csv(RUTA + "olist_order_items_dataset.csv")
items["total"] = items["price"] + items["freight_value"]
print(items[["price", "freight_value", "total"]].head(3))
```

A la izquierda del `=` va el nombre de la columna nueva entre corchetes; a la derecha, el cálculo. Pandas lo hace para las 112.650 filas **de una vez**, sin `for`.

---

2. Fechas: primero hay que convertirlas

Las fechas llegan como **texto** (`'2017-10-02 10:56:33'`) y **no se pueden restar**. Es como un “15” escrito como texto en una planilla: no puedes sumarlo. Hay que **convertir** con `pd.to_datetime` :

```
orders = pd.read_csv(RUTA + "olist_orders_dataset.csv")

orders["compra"] = pd.to_datetime(orders["order_purchase_timestamp"])
orders["entrega"] = pd.to_datetime(orders["order_delivered_customer_date"])

orders["dias_entrega"] = (orders["entrega"] - orders["compra"]).dt.days
print(orders[["compra", "entrega", "dias_entrega"]].head(3))
print("Días promedio:", round(orders["dias_entrega"].mean(), 1))
```

- Restar dos fechas da una **duración** (“8 días 3 horas”); `.dt.days` se queda solo con los días, como número.
- `.mean()` es el promedio; pandas **ignora los vacíos**.
- `round(x, 1)` lo deja con un decimal.
- Donde no hay fecha de entrega queda `NaT` (el “vacío” de las fechas).

El accesor `.dt`: sacar partes de una fecha

```
orders["mes"] = orders["compra"].dt.month
print(orders["mes"].value_counts().sort_index())
```

Expresión	Entrega
`.dt.year`	el año
`.dt.month`	el mes (1 a 12)
`.dt.day`	el día del mes
`.dt.dayofweek`	el día de la semana (0 = lunes)

`value_counts()` ordena por cantidad; `sort_index()` lo reordena de enero a diciembre. Si algo con `.dt` da error, casi siempre **olvidaste convertir** con `to_datetime`.

---

3. Limpiar: vacíos y repetidos

Los datos reales **nunca vienen perfectos**. Hoy vimos dos problemas, con números reales. Y una regla: **cada decisión de limpieza se anota**, porque cambia los resultados.

Vacíos (NaN)

Un vacío es una **celda en blanco**. En pandas se llama **NaN** (“no es un número”).

```
print(orders.isna().sum())
```

`isna()` marca cada celda con verdadero si está vacía; `.sum()` suma los verdaderos por columna. **Es lo primero que corro cuando recibo un archivo nuevo.** Hay casi 3 mil pedidos sin fecha de entrega: algunos están cancelados o aún en camino. *El dato faltante tiene una explicación de negocio.*

```
entregados = orders.dropna(subset=["order_delivered_customer_date"])
print("Antes:", len(orders), "· Después:", len(entregados))     # 99441 → 96476
```

`dropna(subset=[...])` quita las filas vacías **en esas columnas** y devuelve una tabla nueva.

Repetidos

```
reviews = pd.read_csv(RUTA + "olist_order_reviews_dataset.csv")

repetidas = reviews.duplicated(subset="order_id").sum()
print("Reseñas repetidas:", repetidas)                          # 551
reviews_limpias = reviews.drop_duplicates(subset="order_id")
print("Antes:", len(reviews), "· Después:", len(reviews_limpias))
```

- `duplicated(subset="order_id")` marca con verdadero cada fila cuyo `order_id` **ya apareció antes**.
- `drop_duplicates` deja **solo la primera** de cada pedido.

Rellenar en vez de borrar

```
products = pd.read_csv(RUTA + "olist_products_dataset.csv")
categoria = products["product_category_name"].fillna("sin_categoria")
print((categoria == "sin_categoria").sum())                    # 610
```

`fillna` rellena los vacíos con un valor. En vez de perder 610 productos, los marcamos.

¿Borrar o rellenar?

Opción	Ventaja	Riesgo
Borrar (`dropna`)	El dato que queda es real	Pierdes información
Rellenar (`fillna`)	No pierdes filas	Puedes inventar información

No hay receta universal: se decide con **criterio de negocio** y se anota (en la presentación dirás qué limpiaste y por qué).

---

4. La receta de cualquier análisis

Leer → Limpiar → Crear → Responder. Con una pregunta real: *¿cuántos días tarda en llegar un pedido?*

```
orders = pd.read_csv(RUTA + "olist_orders_dataset.csv")                  # 1)
Leer

entregados = orders[orders["order_status"] == "delivered"]               # 2)
Limpiar
entregados = entregados.dropna(subset=["order_delivered_customer_date"]).copy()

compra = pd.to_datetime(entregados["order_purchase_timestamp"])          # 3)
Crear
entrega = pd.to_datetime(entregados["order_delivered_customer_date"])
entregados["dias_entrega"] = (entrega - compra).dt.days

print("Pedidos entregados:", len(entregados))                            # 4)
Responder
print("Promedio de días:", round(entregados["dias_entrega"].mean(), 1))  # 12,1
print("Mínimo y máximo:", entregados["dias_entrega"].min(),
entregados["dias_entrega"].max())
print("Más de 30 días:", (entregados["dias_entrega"] > 30).sum())
```

- `.copy()` crea una **copia independiente**, para poder agregarle columnas sin avisos de pandas.

- `(entregados["dias_entrega"] > 30).sum()` da verdadero/falso por fila y **cuenta los verdaderos**.

Una pregunta de negocio terminó en **cuatro números**: eso es lo que irá en tu presentación.

Errores frecuentes

Qué ves	Por qué pasa	Qué hacer
`TypeError` al restar fechas	Siguen siendo texto	Convierte con `pd.to_datetime` primero
`AttributeError: Can only use .dt accessor...`	La columna no es fecha	Convierte antes de usar `.dt`
`SettingWithCopyWarning`	Modificas una tabla que viene de un filtro	Usa `.copy()` después del filtro
El promedio sale `NaN`	Todos los valores son vacíos	Revisa el filtro y `isna().sum()`
Los resultados cambian sin avisar	Limpiaste sin anotar	Escribe qué quitaste y cuántas filas

---

Practica tú

1. ¿Cuántas filas tiene `reviews` **sin** comentario (`review_comment_message` vacío)?
2. Crea la columna `anio` con el año de compra de cada pedido.
3. ¿Cuántos pedidos tardaron **menos de 5 días**? (usa la tabla `entregados`).
4. ¿Cuál es el estado de pedido más frecuente?

▼ **Ver respuestas**

1. `reviews["review_comment_message"].isna().sum()`
2. `orders["anio"] = orders["compra"].dt.year`
3. `(entregados["dias_entrega"] < 5).sum()`
4. `orders["order_status"].value_counts()` → `delivered`.

Para llevarte

- Columna nueva: `tabla["nombre"] = cálculo`.
- Fechas: **convertir primero** (`to_datetime`), restar, `.dt.days`, `.dt.month`.

- Limpiar: `isna().sum()` para mirar, `dropna` / `fillna` / `drop_duplicates` para actuar, y **anotar** qué se hizo.
- La receta: leer → limpiar → crear → responder.